# 8 QAOA

In the previous chapters we focused on obtaining a Hamiltonian whose ground state represented the solution to our problem. Implicitly or explicitly we referred to quantum annealers as machines for finding the configuration that minimizes the energy.

In this chapter we will show how to move from a problem formulated for quantum annealers to a problem formulated for quantum gate based computers through the *Quantum Approximation Optimization Algorithm* (QAOA), proposed for the first time by Farhi et al. in [41].

We begin by describing the QAOA algorithm, highlighting theoretical advantages and disadvantages with respect to QAC[1]. We will then show how it is possible to translate a problem formulated in QUBO form (Section **??**), obtained at the end of the QA-Prolog pipeline, into a problem suitable for IBM quantum gate based computers.

Finally, we will test the proposed translation and present some experimental results, discussing them.

## 8.1 QAOA

The QAOA algorithm is inspired by QAC; both attempt to find the minimum eigenvalue of a matrix that represents the energy of a physical system [40]. This matrix describes an observable, consequently it is a Hermitian operator and in particular it is the Hamiltonian $\mathbf{H}$ of the system. The smallest eigenvalue of $\mathbf{H}$ is the minimum energy of the system that can be measured; for this reason it is called the ground state energy.

As described in Section **??** QAC is executed on a quantum annealer that starts from a Hamiltonian whose ground state is easy to know and evolves, adiabatically (to remain in the ground state), towards the Hamiltonian with unknown ground state [40]. The evolution time from one Hamiltonian to another can be exponential[41]; moreover, generally the success probability of QAC does not increase as the annealing time increases.

QAOA targets quantum gate based machines and, once its parameters are optimized, the quality of the result increases as the number of iterations, called *levels*, increases.

### 8.1.1 Circuit

The QAOA algorithm begins with a state $|s\rangle$ with maximum superposition. It is possible to obtain this state starting from the state with $n$ *qubits* set

1 Quantum Adiabatic Computing (Section **??**).

to zero ($|0\rangle$) and applying the Hadamard gate (H) to each of them [40]; we denote this operator with $H^{\otimes n}$.

The circuit then consists of $p$ identical levels. Each level is composed of two operators:

- *phase separation operator*: $U_P(\gamma) = e^{-i\gamma \mathbf{H}}$ where $\mathbf{H}$ is the Hamiltonian of the system, and $\gamma \in [0, 2\pi]$ is a parameter;

- *mixing operator*: $U_M(\beta) = e^{-i\beta B}$ where $B$ is the operator that applies the Pauli Gates X to every *qubit*, and $\beta \in [0, \pi]$ is a parameter.

The operator $U_P(\gamma)$ simulates $\mathbf{H}$, while $U_M(\beta)$, which does not commute with $U_P(\gamma)$[2], allows modifying the probability distribution of the output [40].

Using Pauli Gates X is not the only way to construct the mixing operator; in [42, 43] other mixing operators are explored.

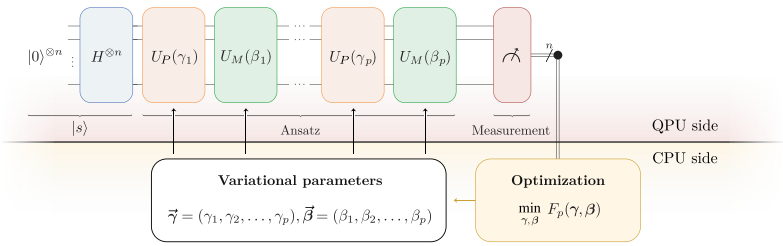



**Figure 8.1**: QAOA scheme [44]

Figure 8.1 shows the general scheme of the QAOA algorithm considering $p$ levels.

Given $\vec{\gamma} = (\gamma_1, \gamma_2, \ldots, \gamma_p)$ and $\vec{\beta} = (\beta_1, \beta_2, \ldots, \beta_p)$, we denote with $\left|\psi(\vec{\gamma}, \vec{\beta})\right\rangle$ the *ansatz* (an estimate of the solution) of QAOA. The ansatz is defined as:

$$\left|\psi(\vec{\gamma}, \vec{\beta})\right\rangle = U_M(\beta_p) U_P(\gamma_p) \cdots U_M(\beta_1) U_P(\gamma_1) |s\rangle . \tag{8.1}$$

The circuit ends with the measurement; the expected value of this measurement is:

$$F(\vec{\gamma}, \vec{\beta}) = \left\langle \psi(\vec{\gamma}, \vec{\beta}) \middle| \mathbf{H} \middle| \psi(\vec{\gamma}, \vec{\beta}) \right\rangle . \tag{8.2}$$

$F(\vec{\gamma}, \vec{\beta})$ represents the expected value of $\mathbf{H}$ in the state $\left|\psi(\vec{\gamma}, \vec{\beta})\right\rangle$ [40]. Once implemented this value is obtained by executing the quantum circuit many times, we will see this later. In addition to the value of the system energy, the measurement of the *qubits* also provides an assignment of the variables that represents a possible solution to the problem.

The QAOA's performance depends on the number of levels; given optimal parameters of QAOA, the algorithm asymptotically converges to optimal solutions as the number of levels grows[41].

2 $U_P(\gamma) \times U_M(\beta) \neq U_M(\beta) \times U_P(\gamma)$.

### 8.1.2 Parameter optimization

QAOA requires tuning the parameters to minimize the expected objective value for a given number of levels. Formally the best parameter ($\vec{\gamma}^*$ and $\vec{\beta}^*$) are defined as:

$$(\vec{\gamma}^*, \vec{\beta}^*) = \arg\min_{\vec{\gamma},\vec{\beta}} F(\vec{\gamma}, \vec{\beta})$$

This parameter tuning problem is nonconvex and NP-hard [40]. Even with one level, the gradient of $F(\vec{\gamma}, \vec{\beta})$ vanishes rapidly; this phenomenon is known as the barren plateau [45, 46]. The barren plateau expands exponentially as the number of qubits grows, making the tuning process even more challenging [46].

The parameter optimization process is executed on a classical CPU rather than on a quantum computer. This makes QAOA a hybrid algorithm [40].

**CONCLUSION:** Despite the difficulty in optimizing the parameters, QAOA algorithm presents numerous interesting aspects and research in this field is very active. In our work it gives us the possibility to change the reference architecture, and attempt to solve a QUBO-formulated problem —obtained with the QA-Prolog pipeline— using quantum gate based computers. In the next sections we will see how to practically implement the QAOA algorithm.

## 8.2 FROM QAC TO QAOA

In the previous section we focused on the theoretical aspects of the QAOA algorithm. In this section instead we show how to use IBM's Qiskit Framework to solve a minimization problem such as those obtained at the end of the QA-Prolog pipeline.

### 8.2.1 Objective

At the end of the QA-Prolog pipeline we obtain a problem in QUBO form: in this representation the variables $x_i$ to be optimized belong to the set $\{0, 1\}$. When we move to the quantum gate based architecture, the measured value of the *qubits* belongs to the set $\{-1, 1\}$. We must therefore move from a representation in QUBO form to a problem in Ising form. Qiskit also requires that the matrix $\mathbf{H}$ is expressed as Pauli operators.

With these steps we have made the output of QA-Prolog compatible with the input of the QAOA algorithm. To complete the implementation we must then optimize the parameters defined in Equations 8.1 and 8.2 in order to minimize the expected value of $\mathbf{H}$. Finally, after finding the optimal parameters we will sample the result of the quantum circuit one last time to obtain the result of our problem.

To summarize, to implement the QAOA algorithm we must:

- move from QUBO form to Ising form;
- express the observable $\mathbf{H}$ using Pauli operators;

- optimize the parameters of Equation **??**;
- sample the optimized quantum circuit.

### 8.2.2 From QUBO to Ising

In Section **??** we saw that a translation from a problem in QUBO form to Ising form always exists. It is possible to implement this translation manually following [**bibid**]; in this case we will use the implementation provided by the Python library `qubovert`.

```
qubo = qubovert.utils.matrix_to_qubo(qubo_mat)
ising = qubovert.utils.qubo_to_quso(qubo)
ising_dict = dict(ising)
ising_dict.pop(())
ising_mat = qubovert.utils.qubo_to_matrix(ising_dict)
```

**Listing 8.1:** From QUBO form to Ising

Listing 8.1 shows how it is possible to move from one formulation to the other using the `qubovert` library:

1. we build a QUBO problem using as input the output of the QA-Prolog pipeline (`qubo_mat`);
2. we transform the QUBO problem into Ising form;
3. we extract the dictionary that assigns the numerical coefficients $a_i$ in Formula **??**;
4. we discard the constant term;
5. we construct the matrix with the weights of the problem in Ising form.

The most interesting aspect of this procedure is the removal of the constant term; this value is used to align the ground state of the problem in QUBO form and Ising form (Equation **??**) to the same energy. Since our objective is to find the ground state, regardless of its value, we can ignore this offset and consider only the coefficients of the variables.

### 8.2.3 Hamiltonian as Pauli operator

Consider the upper triangular Hamiltonian $\mathbf{H}$ that describes our problem in Ising form. Each element belonging to the diagonal $h_{i,i}$ is translated into a gate that measures the Z component of the *qubit* $i$ and applies the identity transformation to all the other *qubits*[3]. The elements $h_{i,j}$[4] are transformed into gates that measure the spin along the Z axis of the *qubits* $i$ and $j$, and apply the identity operator to the other *qubits*.

3 The operator that measures the Z component is $\sigma_z$ (Equation 1.10); this operator is both Hermitian and Unitary, therefore it describes both an observable and a valid quantum gate. The same holds for the identity operator.

4 With $j > i$ because $\mathbf{H}$ is upper triangular.

Following the IBM tutorial [47] we transform the matrix that describes the problem in Ising form into Pauli operators. Listing 8.2 shows how it is possible to automate this rewriting. At line 4 we generate the observable for the elements on the diagonal and at line 6 the observable for the other elements of the matrix.

```
1 def build_paulis(matrix):
2     pauli_list = []
3     for i in range(len(matrix)):
4         pauli_list.append(("Z", [i], matrix[i][i]))
5         for j in range(i+1, len(matrix)):
6             pauli_list.append(("ZZ", [i, j], matrix[i][j]))
7     return pauli_list
```

**Listing 8.2:** Genearating Pauli operator

We can finally construct a single observable starting from the list of Pauli operators that we have just built. To do this we use a library function provided by Qiskit. Listing 8.3 shows how to obtain the cost function, where `pauli_list` are the Pauli operators that we generated with Listing 8.2, and `n_qubits` is the number of *qubits* that describe our problem.

```
cost_hamiltonian = SparsePauliOp.from_sparse_list(pauli_list, n_qubits)
```

**Listing 8.3:** Defining cost function

### 8.2.4 Final circuit

As seen in Section 8.1.1 the QAOA algorithm does not directly use the cost function, but the parametric gates $U_P(\gamma)$ and $U_M(\beta)$ repeated a certain number of times.

Qiskit provides the programmer with a constructor to generate the quantum circuit that implements the QAOA algorithm; this constructor allows specifying:

- cost function: the one obtained in Listing 8.3;
- number of repetitions;
- initial state: in the form of a quantum circuit. If no initial state is specified, the Hadamard gate is applied to each *qubit*;
- mixing operator: this parameter can be specified as a base operator or as a quantum circuit. If no parameter is specified the Not gate **??** is applied.

The circuit ends by measuring the value of the *qubits*. Listing 8.4 shows the construction of the circuit.

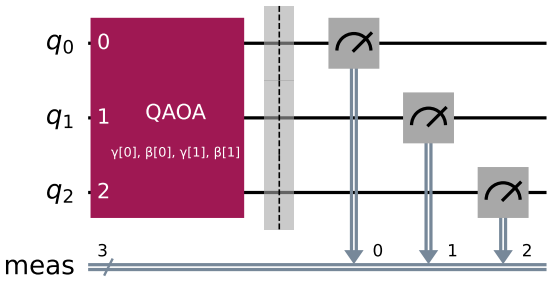
```
1 qc = QAOAAnsatz(cost_operator=cost_funcrion, reps=n_reps)
2 qc.measure_all()
```

**Listing 8.4:** Circuit construction

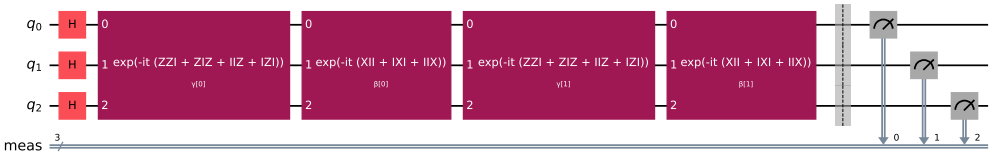
**EXAMPLE:** Consider a problem described by three *qubits* and the QAOA algorithm with two levels. Figure 8.2 shows: in a the matrix that describes the problem; in b the circuit in compact form; in c the circuit expanded to observe how the coefficients of the matrix have been translated into observables.

$$\mathbf{H} = \begin{pmatrix} 1 & 0 & 1 \\ 0 & 1 & 1 \\ 0 & 0 & 0 \end{pmatrix}$$

**(a)** Hamiltonian matrix




**(b)** Compact form




**(c)** Expanded form

**Figure 8.2:** QAOA circuit

By carefully examining the operators that constitute the gate $U_P(\gamma)$ (first and third block in Figure 8.2c) we can observe that the order of the qubits is inverted. For example, the operator IIZ represents the element of $\mathbf{H}$ in position $(0, 0)$: I is the identity gate applied to the third and second *qubits*, and Z is the Pauli Gate Z applied to the first *qubit*. The operator ZZI represent the element in position $(1, 2)$. This observation must be kept in mind when reading the results of the circuit.

### 8.2.5 Parameter optimization

We define a procedure to obtain the cost $F(\vec{\gamma}, \vec{\beta})$ (Equation 8.2) as the expected value of the result of executing the quantum circuit defined earlier. This cost must be minimized by optimizing the parameters $\vec{\gamma}$ and $\vec{\beta}$. Following [47], we use the Python library `scipy` to minimize $F(\vec{\gamma}, \vec{\beta})$.

Listing 8.5 shows how to obtain $F(\vec{\gamma}, \vec{\beta})$ as an expected value:

- at line 5 we create a *primitive unified block* (`pub`): a triple containing the circuit to execute (`ansatz`), the observable to measure (`isa_hamiltonian`) and the circuit parameters (`params`);
- at line 6 we estimate $F(\vec{\gamma}, \vec{\beta})$ by executing multiple runs of the circuit[5];
- from line 7 to 9 we retrieve and return the result.

Listing 8.6 shows the minimization process: we create an estimator that executes 1000 runs of the circuit (lines 2 and 3) and use the library function

5 The shot's number is an attribute of the `estimator` object that we define in Listing 8.6.

```python
def cost_func_estimator(params, ansatz, hamiltonian, estimator):

    # transform the observable defined on virtual qubits to
    # an observable defined on all physical qubits

    isa_hamiltonian = hamiltonian.apply_layout(ansatz.layout)
    pub = (ansatz, isa_hamiltonian, params)
    job = estimator.run([pub])
    results = job.result()[0]
    cost = results.data.evs
    objective_func_vals.append(cost)


    return cost
```

**Listing 8.5:** $F(\vec{\gamma}, \vec{\beta})$ estimation

```python
1  with Session(backend=backend) as session:
2      estimator = Estimator(mode=session)
3      estimator.options.default_shots = 1000
4
5      result = minimize(
6          cost_func_estimator,
7          init_params,
8          args=(candidate_circuit, cost_hamiltonian, estimator),
9          method="COBYLA",
10         tol=1e-2,
11     )
12 optimized_circuit = candidate_circuit.assign_parameters(result.x)
```

**Listing 8.6:** $\vec{\gamma}$ and $\vec{\beta}$ optimization

`scipy.optimize.minimize`[6] to minimize $F(\vec{\gamma}, \vec{\beta})$ by optimizing the parameters $\vec{\gamma}$ and $\vec{\beta}$. Finally we create the optimized circuit by assigning to the parametric circuit the value of the optimized parameters (line 12). In this listing, `candidate_circuit` is the transpiled[7] version of the circuit `qc` obtained in Listing 8.4.

### 8.2.6 Retrieving the results

Now that we have optimized the value of the parameters we can, one last time, perform sampling of the quantum circuit. This time we are not interested in estimating the minimum value of $F(\vec{\beta}, \vec{\gamma})$, but in determining the value of the *qubits* that generate this value.

In Listing 8.7 we create a `Sampler` (line 1) that runs the optimized circuit 10000 times (line 4). In this case we use a `Sampler`, and not an `Estimator` as in Listing 8.6, because we are interested in the value of the *qubits* rather than the expected value of an observable. At line 5 we retrieve the results.

6 Its documentation is available at https://docs.scipy.org/doc/scipy/reference/generated/scipy.optimize.minimize.html.
7 Section 5.2.2

```
sampler = Sampler(mode=backend)
sampler.options.default_shots = 10000
pub = (optimized_circuit,)
job = sampler.run([pub], shots=int(1e4))
counts_bin = job.result()[0].data.meas.get_counts()
```

**Listing 8.7:** Circuit sampling

## 8.3 EXPERIMENTS

We apply the QAOA algorithm to solve a problem in QUBO form obtained with the QA-Prolog pipeline.

### 8.3.1 Problem

Consider the satisfiability problem $y = x_1 \wedge \neg(x_2 \vee x_3)$ (Equation 6.1). This problem is well suited for the application of QAOA because we can read the solution directly from the value of the *qubits*[8].

As shown in Example 6.2.3, starting from Prolog code to solve the satisfiability problem introduces a significant amount of overhead. To minimize the number of *qubits* required to represent the problem we follow only part of the pipeline, starting from the Verilog code of Example 6.2.2.

The QUBO problem that we obtain is represented by an $11 \times 11$ matrix, this means that we need eleven *qubits* to encode our problem.

$$\mathbf{H} = \begin{pmatrix} 4 & 2 & -4 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & -8 \\ 0 & 4 & -4 & 0 & -8 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 2 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 2 & 4 & 0 & 0 & -8 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 2 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 6 & 2 & -4 & -8 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 6 & -4 & 0 & -8 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 6 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 4 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 4 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 4 \end{pmatrix}$$

**(a)** QUBO Hamiltonian

$$\mathbf{H} = \begin{pmatrix} 0.5 & 0.5 & -1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & -2 \\ 0 & 0.5 & -1 & 0 & -2 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & -2 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & -0.5 & 0.5 & -1 & -2 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & -0.5 & -1 & 0 & -2 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{pmatrix}$$

**(b)** Ising Hamiltonian

**Figure 8.3:** Problem's Hamiltonian

Figure 8.3 shows the matrix representation of the problem both in QUBO and Ising form. The two representations have ground states with different values, but they correspond to the same configuration of the *qubits*.

### 8.3.2 Execution

To solve the satisfiability problem we construct a QAOA with 10 levels. By increasing the number of levels, the algorithm converges increasingly well to the optimal solution.

However, increasing the number of levels also leads to longer execution time and, if executed on quantum hardware (and not through simulators),

8 We will discuss the possibility of using QA-Prolog to make the output of QAOA readable in **??**.

the deeper the circuit, the more one encounters the phenomenon of decoherence [**bibid**]. Moreover, the number of parameters to optimize is equal to twice the number of levels, therefore increasing the number of levels also makes the parameter optimization more complex[9].

Figure 8.4 shows the parameter optimization process: as better parameters are found the estimated cost decreases. In this case the parameters were optimized after approximately 130 iterations. Each of these iterations involves 1000 runs of the quantum circuit.

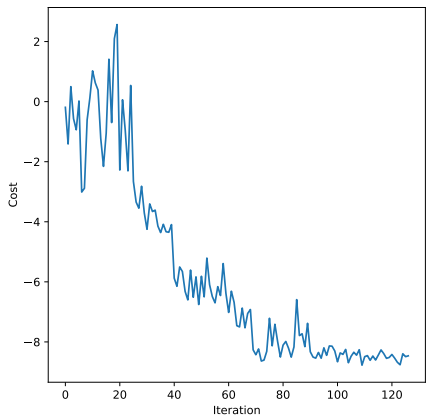
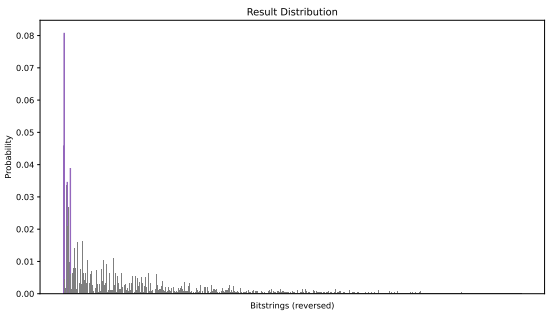
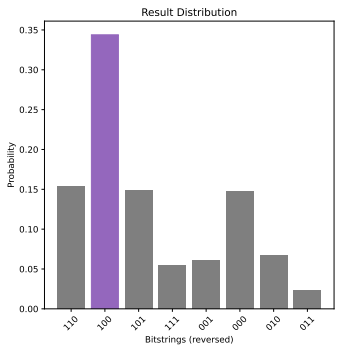



**Figure 8.4:** Cost minimization

Once the parameters have been found we can sample the optimized circuit to obtain the answer to our problem. Figure 8.5a shows the result of the sampling (10000 runs of the quantum circuit), highlighting in purple the four most frequent results.




**(a)** All *qubits*




**(b)** Interesting *qubits*

**Figure 8.5:** Circuit sampling

The histogram in Figure 8.5a is difficult to interpret; moreover it shows the result of all eleven *qubits* that form the polynomial to be minimized. We are interested only in the value of the variables $x_1$, $x_2$ and $x_3$, the others are ancillary. Figure 8.5b shows the sampling result where we consider only the variables of interest. These were extracted by checking which *qubits* had been mapped by QMASM to the variables $x_1$, $x_2$ and $x_3$, and considering that the obtained bitstring must be read in reverse.

The obtained result is the expected one; in fact we obtained the string 1 0 0 as the most probable sample. These values must be associated respectively with the variables $x_1$, $x_3$ and $x_2$, and we can easily verify that this assignment satisfies Equation 6.1.

9 As sed in Section 8.1.2

### 8.3.3 Results

Both the sampling process and the parameter optimization process introduce stochastic components into the execution of QAOA. We therefore want to verify, by executing the parameter optimization and the final sampling multiple times, if we always obtain the expected result[10].

Since the execution time of each experiment needs to be taken into consideration, the number of experiments performed is limited and does not allow reliable statistical analysis. However, it is sufficient to show how the QAOA algorithm is not always able to find the correct answer to the minimization problem.

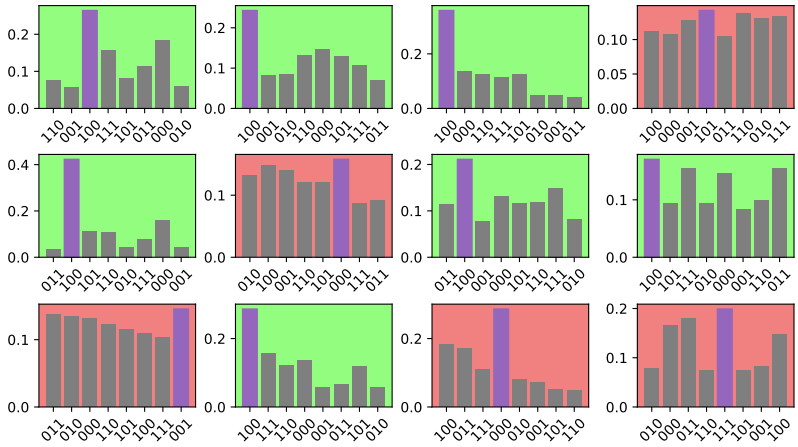


**Figure 8.6:** Multiple experiments' result

Figure 8.6 shows the results (of the relevant *qubits*) of 12 repetitions of the experiment. In the image, all the cases in which the correct assignment of the variables $x_1$, $x_2$ and $x_3$ was identified are highlighted with a green. The cases, in which the most probable result is incorrect, are highlighted in red.

The number of experiments is very low and advanced statistical analyses would not be reliable. Nevertheless, we can make some qualitative observations. Typically, in the cases in which the correct solution is found, the probability of the latter is noticeably higher than that of the other results. When instead the solution is incorrect, the different results are distributed in a rather uniform way in over half of the cases.

## 8.4 CONCLUSION

In this chapter we presented a new algorithm for solving a minimization problem in QUBO form. From a theoretical point of view, we saw that QAOA guarantees a monotonic improvement in performance as the number of levels increases (provided that optimal parameters have been found), while QAC does not guarantee this monotonicity when increasing the an-

10 The code that automates the execution of the experiments is available at https://github.com/DavideCamino/Thesis/blob/main/Code/3_QAUA/circ_sat/circ_sat_qaoa_multiple_run.ipynb.

nealing time. We then presented the critical aspects of QAOA, highlighting how the problem of finding optimal parameters is, in general, NP-Hard.

The discussion then moved to the practical implementation, and we saw how to use the Qiskit Framework to solve on IBM quantum gate based machines a problem originally in QUBO form. We presented Python scripts to automate the translation, we saw how to retrieve the results and how to interpret them. Finally, we discussed the quality of the solutions provided by QAOA by repeating the parameter tuning and the sampling of the optimized circuit, discussing the obtained results.

In the previous chapter we showed how to add a rewriting step upstream of the QA-Prolog pipeline. This chapter can also be considered as a further rewriting step of a problem into different but equivalent formats; this time downstream of the pipeline we took the output provided by QMASM and rewrote it into a format compatible with QAOA.

# BIBLIOGRAPHY


[1] Barry Smith. "Ontology". In: (2012).

[2] Gian Piero Zarri. "Ontologies and Their Practical Implementation". In: *Encyclopedia of Database Technologies and Applications*. IGI Global Scientific Publishing, 2005, pp. 438–449.

[3] Thomas R Gruber. "A translation approach to portable ontology specifications". In: *Knowledge acquisition* 5.2 (1993), pp. 199–220.

[4] Marco Fossati, Emilio Dorigatti, and Claudio Giuliano. "N-ary relation extraction for simultaneous T-Box and A-Box knowledge base augmentation". In: *Semantic Web* 9.4 (2018), pp. 413–439.

[5] Giuseppe De Giacomo, Maurizio Lenzerini, et al. "TBox and ABox reasoning in expressive description logics." In: *KR* 96.316-327 (1996), p. 10.

[6] Markus Krötzsch, Frantisek Simancik, and Ian Horrocks. "A description logic primer". In: *arXiv preprint arXiv:1201.4089* (2012).

[7] Pascal Hitzler et al. *OWL 2 Web Ontology Language Primer (Second Edition)*. W3C Recommendation REC-owl2-primer-20121211. World Wide Web Consortium (W3C), Dec. 2012. URL: https://www.w3.org/TR/2012/REC-owl2-primer-20121211/.

[8] Pascal Hitzler. "A review of the semantic web field". In: *Communications of the ACM* 64.2 (2021), pp. 76–83.

[9] Stefano Borgo et al. "DOLCE: A descriptive ontology for linguistic and cognitive engineering". In: *Applied ontology* 17.1 (2022), pp. 45–69.

[10] Boris Motik, Peter F. Patel-Schneider, and Bernardo Cuenca Grau. *OWL 2 Web Ontology Language: Direct Semantics (Second Edition)*. W3C Recommendation REC-owl2-direct-semantics-20121211. World Wide Web Consortium (W3C), Dec. 2012. URL: https://www.w3.org/TR/2012/REC-owl2-direct-semantics-20121211/.

[11] Ilianna Kollia, Birte Glimm, and Ian Horrocks. "Query answering over SROIQ knowledge bases with SPARQL". In: *Proceedings of the 24th International Workshop on Description Logics, Barcelona, Spain*. 2011, pp. 13–16.

[12] Franz Baader. *The description logic handbook: Theory, implementation and applications*. Cambridge university press, 2003.

[13] Birte Glimm et al. "HermiT: an OWL 2 reasoner". In: *Journal of automated reasoning* 53.3 (2014), pp. 245–269. URL: http://www.hermit-reasoner.com/.

[14] Scott Pakin. "Performing fully parallel constraint logic programming on a quantum annealer". In: *Theory and Practice of Logic Programming* 18.5-6 (2018), pp. 928–949.

[15] William M Kaminsky, Seth Lloyd, and Terry P Orlando. "Scalable superconducting architecture for adiabatic quantum computation". In: *arXiv preprint quant-ph/0403090* (2004).

[16] Los Alamos National Laboratory / Scott Pakin. *QA-Prolog: Quantum Annealing Prolog*. Accessed: 2026-02-13, GitHub repository. URL: https://github.com/lanl/QA-Prolog.

[17] Umer Farooq, Zied Marrakchi, and Habib Mehrez. "FPGA architectures: An overview". In: *Tree-Based Heterogeneous FPGA Architectures: Application Specific Exploration and Optimization* (2012), pp. 7–48.

[18] William F Clocksin and Christopher S Mellish. *Programming in PROLOG*. Springer Science & Business Media, 2003.

[19] Patrick Blackburn, Johan Bos, and Kristina Striegnitz. *Learn Prolog Now! – Free Online Version*. Accessed: 2026-02-13. 2012. URL: https://lpn.swi-prolog.org/lpnpage.php?pageid=online.

[20] Robert Kowalski and Steve Smoliar. "Logic for problem solving". In: *ACM SIGSOFT Software Engineering Notes* 7.2 (1982), pp. 61–62.

[21] Christopher John Hogger. *Introduction to logic programming*. Academic Press Professional, Inc., 1984.

[22] SWI-Prolog Developers. *SWI-Prolog Reference Manual*. Accessed: 2026-02-20. SWI-Prolog. URL: https://www.swi-prolog.org/pldoc/doc_for?object=manual.

[23] Jan Wielemaker et al. "SWI-Prolog". In: *Theory and Practice of Logic Programming* 12.1-2 (2012), pp. 67–96. ISSN: 1471-0684.

[24] Markus Triska. "The finite domain constraint solver of SWI-Prolog". In: *International Symposium on Functional and Logic Programming*. Springer. 2012, pp. 307–316.

[25] Markus Triska. *clpfd: Constraint Logic Programming over Finite Domains*. https://github.com/triska/clpfd. GitHub repository, accessed 2026-02-20.

[26] Markus Triska. *clpz: Constraint Logic Programming over Integers*. https://github.com/triska/clpz. GitHub repository, accessed 2026-02-20.

[27] Roshan P James, Gerardo Ortiz, and Amr Sabry. "Quantum computing over finite fields". In: *arXiv preprint arXiv:1101.3764* (2011).

[28] Mohamed W Hassan, Scott Pakin, and Wu-chun Feng. "C to D-wave: a high-level C compilation framework for quantum Annealers". In: *2019 IEEE High Performance Extreme Computing Conference (HPEC)*. IEEE. 2019, pp. 1–8.

[29] Scott Pakin. "A quantum macro assembler". In: *2016 IEEE High Performance Extreme Computing Conference (HPEC)*. IEEE. 2016, pp. 1–8.

[30] Michael R Garey and David S Johnson. *Computers and intractability*. Vol. 29. wh freeman New York, 2002.

[31] Mike Gordon. "The semantic challenge of Verilog HDL". In: *Proceedings of tenth annual IEEE symposium on logic in computer science*. IEEE. 1995, pp. 136–145.

[32] Scott Pakin. "Targeting classical code to a quantum annealer". In: *Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems*. 2019, pp. 529–543.

[33] Clifford Wolf, Johann Glaser, and Johannes Kepler. "Yosys-a free Verilog synthesis suite". In: *Proceedings of the 21st Austrian Workshop on Microelectronics (Austrochip)*. Vol. 97. 2013.

[34] Robert Brayton and Alan Mishchenko. "ABC: An academic industrial-strength verification tool". In: *International Conference on Computer Aided Verification*. Springer. 2010, pp. 24–40.

[35] Hongyu Zhang, Yuan-Fang Li, and Hee Beng Kuan Tan. "Measuring design complexity of semantic web ontologies". In: *Journal of Systems and Software* 83.5 (2010), pp. 803–814.

[36] Markus Triska. "Boolean constraints in SWI-Prolog: A comprehensive system description". In: *Science of Computer Programming* 164 (2018), pp. 98–115.

[37] Mark A Musen. "The protégé project: a look back and a look forward". In: *AI matters* 1.4 (2015), pp. 4–12.

[38] Ken Samuel et al. "Translating owl and semantic web rules into prolog: Moving toward description logic programs". In: *Theory and Practice of Logic Programming* 8.3 (2008), pp. 301–322.

[39] G Filatrella, P Romano, et al. "ELABORAZIONE STATSTICA DEI DATI SPERIMENTALI con elementi di laboratorio". In: EdiSES, 2009, pp. 135–136. ISBN: 978-88-7959-513-1.

[40] Ramin Fakhimi and Hamidreza Validi. "Quantum approximate optimization algorithm (QAOA)". In: *Encyclopedia of Optimization*. Springer, 2023, pp. 1–7.

[41] Edward Farhi, Jeffrey Goldstone, and Sam Gutmann. "A quantum approximate optimization algorithm". In: *arXiv preprint arXiv:1411.4028* (2014).

[42] Zhihui Wang et al. "XY mixers: Analytical and numerical results for the quantum alternating operator ansatz". In: *Physical Review A* 101.1 (2020), p. 012320.

[43] Stuart Hadfield et al. "From the quantum approximate optimization algorithm to a quantum alternating operator ansatz". In: *Algorithms* 12.2 (2019), p. 34.

[44] Kostas Blekos et al. "A review on quantum approximate optimization algorithm and its variants". In: *Physics Reports* 1068 (2024), pp. 1–66.

[45] Andrew Arrasmith et al. "Effect of barren plateaus on gradient-free optimization". In: *Quantum* 5 (2021), p. 558.

[46] Jarrod R McClean et al. "Barren plateaus in quantum neural network training landscapes". In: *Nature communications* 9.1 (2018), p. 4812.

[47] IBM Quantum. *Quantum Approximate Optimization Algorithm*. IBM Quantum documentation tutorial, accessed 2026-03-09. URL: https://quantum.cloud.ibm.com/docs/en/tutorials/quantum-approximate-optimization-algorithm.